

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce Department of Information Technology (B.Sc.I.T Semester VI)
Business Analytics
Practical I

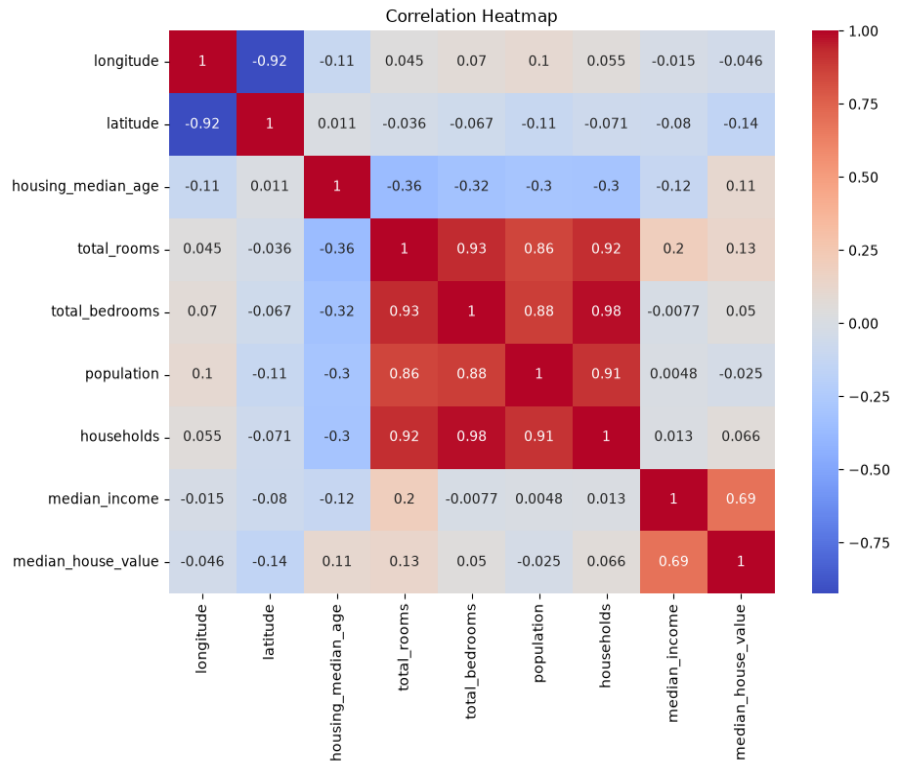
Roll No. : T052	Name:Divyanjali yadav
Class : TYIT	Date/Time of Submission : 14/08/2026

3. Finding Relationships Among Variables:

Aim:a). Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.

```
Code:import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load Dataset
df = pd.read_csv("housing.csv")
# Display First Five Records
print(df.head())
# Display Numerical Columns
print(df.select_dtypes
(include="number").columns)
# Calculate Correlation Matrix
correlation = df.corr
(numeric_only=True)
print(correlation)
# Display Correlation Heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()
```

Output:-



```

longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0   -122.23    37.88           41           880           129.0
1   -122.22    37.86           21          7099          1106.0
2   -122.24    37.85           52          1467           190.0
3   -122.25    37.85           52          1274           235.0
4   -122.25    37.85           52          1627           280.0

population  households  median_income  median_house_value  ocean_proximity
0           322         126         8.3252         452600         NEAR BAY
1          2401        1138         8.3014         358500         NEAR BAY
2           496         177         7.2574         352100         NEAR BAY
3           558         219         5.6431         341300         NEAR BAY
4           565         259         3.8462         342200         NEAR BAY
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='str')
```

```

longitude  longitude  latitude  housing_median_age  total_rooms  \
longitude    1.000000  -0.924664  -0.108197    0.044568  \
latitude    -0.924664    1.000000    0.011173   -0.036100  \
housing_median_age  -0.108197    0.011173    1.000000   -0.361262  \
total_rooms        0.044568  -0.036100    -0.361262    1.000000  \
total_bedrooms     0.069608  -0.066983   -0.320451    0.930380  \
population         0.099773  -0.108785   -0.296244    0.857126  \
households         0.055310  -0.071035   -0.302916    0.918484  \
...
population         -0.024650
households         0.065843
median_income      0.688075
median_house_value  1.000000
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

b. Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).

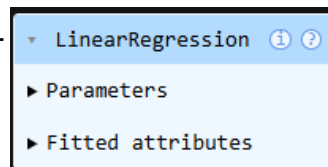
Code: # Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load Dataset
df = pd.read_csv("housing.csv")
# Select Independent and Dependent Variables
X = df[["median_income"]]
y = df["median_house_value"]
# Split Dataset into Training and Testing Data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Create Linear Regression Model
model = LinearRegression()
# Train the Model
model.fit(X_train, y_train)
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, y)
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
```

Output:-



```
# Predict House Values
y_pred = model.predict(X_test)
# Display First 10 Predictions
print("Predicted Values:")
print(y_pred[:10])
# Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
# Calculate R2 Score
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
# Scatter Plot with Regression Line
plt.figure(figsize=(8, 5))
```

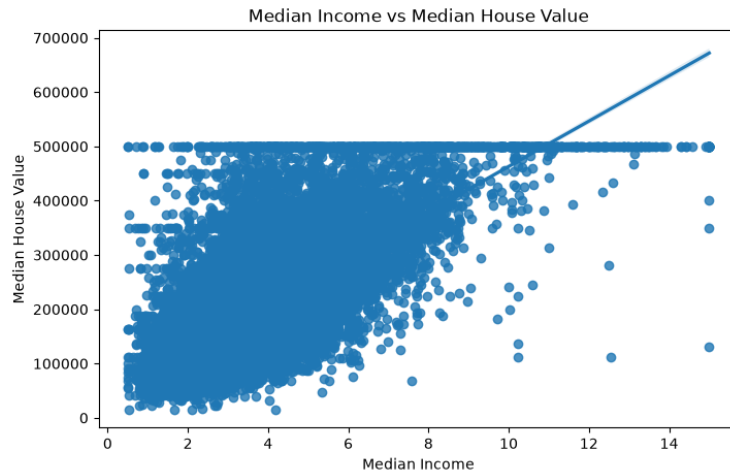
Output:-

```
Coefficient: 41793.849201896286
Intercept: 45085.5767032679
Predicted Values:
[115349.3959815  150878.34718803 190532.35131079 284881.96588407
 200767.66498033 242131.03753545 257561.32666079 199338.31533763
 245846.5107295  384167.43404809]
Mean Squared Error: 7090457293.46965
```

Output:-

```
R2 Score: 0.4589126439756298
```

```
sns.regplot(
    x="median_income",
    y="median_house_value",
    data=df
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median Income vs Median House Value")
plt.show()
```



Additional Questions on housing.csv (Must include in Print document)

Q.1 Calculate the complete correlation matrix for all numerical variables.

Identify:

The strongest positive correlation.

The strongest negative correlation.

The weakest correlation.

Display the results.

```
Code:import pandas as pd
import numpy as np
df = pd.read_csv("housing.csv")
numerical_df = df.select_dtypes(include="number")
correlation_matrix = numerical_df.corr()
print("Complete Correlation Matrix:")
print(correlation_matrix)
upper = correlation_matrix.where(
    np.triu(np.ones(correlation_matrix.shape), k=1).astype(bool)
)
correlations = upper.stack()
print("\nStrongest Positive Correlation:")
print(correlations.idxmax(), "=", correlations.max())
print("\nStrongest Negative Correlation:")
print(correlations.idxmin(), "=", correlations.min())
```

Complete Correlation Matrix:

	longitude	latitude	housing_median_age	total_rooms	\
longitude	1.000000	-0.924664	-0.108197	0.044568	
latitude	-0.924664	1.000000	0.011173	-0.036100	
housing_median_age	-0.108197	0.011173	1.000000	-0.361262	
total_rooms	0.044568	-0.036100	-0.361262	1.000000	
total_bedrooms	0.069608	-0.066983	-0.320451	0.930380	
population	0.099773	-0.108785	-0.296244	0.857126	
households	0.055310	-0.071035	-0.302916	0.918484	
median_income	-0.015176	-0.079809	-0.119034	0.198050	
median_house_value	-0.045967	-0.144160	0.105623	0.134153	

	total_bedrooms	population	households	median_income	\
longitude	0.069608	0.099773	0.055310	-0.015176	
latitude	-0.066983	-0.108785	-0.071035	-0.079809	
housing_median_age	-0.320451	-0.296244	-0.302916	-0.119034	
total_rooms	0.930380	0.857126	0.918484	0.198050	
total_bedrooms	1.000000	0.877747	0.979728	-0.007723	
population	0.877747	1.000000	0.907222	0.004834	
households	0.979728	0.907222	1.000000	0.013033	
median_income	-0.007723	0.004834	0.013033	1.000000	
median_house_value	0.049686	-0.024650	0.065843	0.688075	

```
median_house_value
longitude          -0.045967
...
('longitude', 'latitude') = -0.9246644339150368

Weakest Correlation:
('population', 'median_income') = 0.004834345627653126
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
print("\nWeakest Correlation:")
print(correlations.abs().idxmin(), "=", correlations.loc[correlations.abs().idxmin()])
```

Q.2 Build a regression model using:

X =median_income

X=housing_median_age

X=total_rooms

X=population

X=households

Y =median_house_value

Evaluate the model using:

- **R2**

- **Mean Squared Error**

Compare this model with the simple regression model using only median_income.

```
Code:-import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
# Load dataset
df = pd.read_csv("housing.csv")
# Remove missing values
df = df.dropna()
# Multiple Regression
X = df[
    [
        "median_income",
        "housing_median_age",
        "total_rooms",
        "population",
        "households"
    ]
]
Y = df["median_house_value"]
# Split data
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)
# Create model
multiple_model = LinearRegression()
# Train model
multiple_model.fit(X_train, Y_train)
# Prediction
Y_pred_multiple = multiple_model.predict(X_test)
# Evaluation
```

```

multiple_r2 = r2_score(Y_test, Y_pred_multiple)
multiple_mse = mean_squared_error(Y_test, Y_pred_multiple)
print("MULTIPLE REGRESSION MODEL")
print("-----")
print("R2 Score:", multiple_r2)
print("Mean Squared Error:", multiple_mse)
# Simple Regression
X_simple = df[["median_income"]]
Y_simple = df["median_house_value"]
X_train_s, X_test_s, Y_train_s, Y_test_s = train_test_split(
    X_simple, Y_simple, test_size=0.2, random_state=42
)
simple_model = LinearRegression()
simple_model.fit(X_train_s, Y_train_s)
Y_pred_simple = simple_model.predict(X_test_s)
simple_r2 = r2_score(Y_test_s, Y_pred_simple)
simple_mse = mean_squared_error(Y_test_s, Y_pred_simple)
print("\nSIMPLE REGRESSION MODEL")
print("-----")
print("R2 Score:", simple_r2)
print("Mean Squared Error:", simple_mse)
# Comparison
print("\nMODEL COMPARISON")
print("-----")
print("Multiple Regression R2:", multiple_r2)
print("Simple Regression R2:", simple_r2)
print("Multiple Regression MSE:", multiple_mse)
print("Simple Regression MSE:", simple_mse)

```

```

MULTIPLE REGRESSION MODEL
-----
R2 Score: 0.5689722351518441
Mean Squared Error: 5894382608.105016

SIMPLE REGRESSION MODEL
-----
R2 Score: 0.47196228574894983
Mean Squared Error: 7221011204.235033

MODEL COMPARISON
-----
Multiple Regression R2: 0.5689722351518441
Simple Regression R2: 0.47196228574894983
Multiple Regression MSE: 5894382608.105016
Simple Regression MSE: 7221011204.235033

```

Q.3 Find the correlation between:

- total_rooms
- median_house_value

Code:-import pandas as pd

Load dataset

df = pd.read_csv("housing.csv")

Calculate correlation

correlation = df["total_rooms"].corr(df["median_house_value"])

print("Correlation between total_rooms and median_house_value:")

print(correlation)

Output:

```

Correlation between total_rooms and median_house_value:
0.1341531138065631

```